

VIXOR-MXB-001

VIXOR ENGINEERING BIBLE

إنجيل موكسي

التوثيق الكامل لفريق التداول الذكي MOXI — من الشخصية إلى التنفيذ

جدول المحتويات

نظرة عامة على MOXI

شخصية MOXI ونظام Avatar

محرك السياق (Context Engine)

نظام الأدوات (Tool System)

مولّد الأوامر (Prompt Builder)

الاستجابة الاستباقية (Proactive Insights)

البث المباشر والذكاء الاصطناعي

قاعدة بيانات MOXI

واجهات MOXI في المنتج

خريطة التتبع: MOXI → الكود

نظرة عامة على MOXI

MOXI ليس روبوت محادثة عاديًا، ولا مجرد مساعد ذكي يردّ على الأسئلة. إنه **رفيق التداول الاستباقي** الذي يعيش داخل منظومة الطرفية الذكية لتداول عملات الميم على سولانا. صُمِّمَ MOXI ليكون الوجه البشري للذكاء الاصطناعي في فيكسور: يفهم سياق المفضلاته، يراقب محفظته، ويبادر بالتحذيرات قبل أن يُطرح السؤال.

يُمثِّل MOXI نقلة نوعية في كيفية تفاعل المتداول مع البيانات. بدلاً من تصفّح سبعة لوحات مختلفة وقراءة إشارات وتحليلات MOXI كل السياق في لحظة واحدة ويقدم رؤية متماسكة. يشمل ذلك محفظة المستخدم، استراتيجيته الحالية، إشارات التداول والعملات المباشرة، قائمة المراقبة، والأحداث القادمة — كلها تُجمع بشكل متوازٍ ثم تُقدَّم كجملة واحدة ذات معنى.

يتميّز MOXI بثلاث خصائص جوهرية: **الاستباقية** حيث يراقب المراكز المفتوحة ويبادر بتنبيهات دون طلب، **الشخصية القابلة** حيث يمكن لكل مستخدم ضبط اسم وشخصية وطريقة تواصل MOXI، و**الذكاء العميق** حيث يستخدم سلسلة مزوّدي الذكاء الاصطناعي (openai → groq → anthropic →) مع نظام أدوات متكامل لتحليل أزواج التداول وإنشاء التنبيهات.

التصميم الجوهري

MOXI يتحوّل من مستجيب سلبي إلى شريك نشط في التداول. لا ينتظر المستخدم ليسأل — بل يراقب، يحلّل، ويُنبه. هذه هي الفلسفة التي عن كل مساعد تداول آخر.

من الناحية التقنية، يعمل MOXI داخل نطاق `src/domains/moxi/` كـ مجال مستقل يحتوي على ثمانية ملفات أساسية تغطي تعريف الأنواع إلى توليد التنبيهات الاستباقية. يتكامل مع البنية التحتية لفيكسور عبر `supabase` لاسترجاع بيانات `LLMRouter` لتوجيه الطلبات لمزوّدي الذكاء الاصطناعي، و `processWithAgent` لاكتشاف نية استخدام `/api/copilot-stream` للبيث المباشر عبر SSE.

تدفع العمل يبدأ عندما يُرسل المستخدم رسالة إلى الدالة الخادمية `askMoxi`. تُطبّق حدود المعدّل (25 طلب/دقيقة/مستخدم) محرك السياق الذي يجلب تسعة مصادر بيانات بالتوازي. بعدها يمرّ الطلب عبر طبقة الذكاء حيث يُكتشف إن كان يحتاج أداة أم لا. مزوّد الذكاء الاصطناعي المناسب مع السياق الكامل. النتيجة تُبثّ مباشرة عبر SSE إلى واجهة المستخدم.

شخصية MOXI ونظام Avatar

كل مستخدم في فيكسور يحصل على نسخته الخاصة من MOXI — رفيق يحمل اسمًا وشخصية ومظهرًا فريدًا. نظام الشخصية عميقًا يبدأ من الاسم وصولاً إلى نمط التواصل وخبيرة الذكاء الاصطناعي. الهدف هو جعل MOXI يشعر كشريك حقيقي يفهم أسلوب التداول، وليس مجرد أداة جامدة.

التعريف الأساسي للشخصية يُخزّن في قاعدة البيانات ضمن جدول `moxi_personas`، ويُعبّر عنه برمجياً عبر نوع `Persona` الذي يحتوي على: الاسم، الشخصية النصية، الخبرات كمصفوفة JSONB، نمط التواصل (رسمي/عادي/مختلط)، متغير المظهر، مع غير القابل للاستبدال إن وُجد، وعلامة التخصيص.

نوع MoxiPersona — تعريف TypeScript

```

MoxiPersona = {
  name: string; // اسم الشخصية، الافتراضي 'MOXI'
  description: string; // وصف نصي للشخصية
  interests: string[]; // مجالات الخبرة
  communication_style: 'formal' | 'casual' | 'mixed';
  avatar_variant: MoxiAvatarVariant;
  nft_id?: string; // إن وُجد NFT معرف
  is_customized: boolean; // هل حُصّصت الشخصية؟
};
```

ثمانية متغيرات مظهر (`MoxiAvatarVariant`) تُتيح للمستخدم اختيار الهوية البصرية التي تناسبه. كل متغير يحمل دلالة بصرية مختلفة:

المتغير	الدلالة	الاستخدام المثالي
default	المظهر الأساسي — هويّة فيكسور	الخيار الافتراضي لجميع المستخدمين
bull	ثور صاعد — حماس وثقة	المتداولون المتفائلون الذين يبحثون عن فرص الشراء
bear	دبّ هابط — حذر ومحلّل	المتداولون المحافظون الذين يفضّلون التحذيرات المبكرة
crystal	بلّورة صافية — وضوح وبصيرة	المستخدمون الذين يفضّلون التحليلات الموضوعية الخالية من العاطفة
flame	لهب متقدّد — سرعة وحسم	المتداولون السريعون (Scalpers) الذين يحتاجون قرارات فورية
ocean	محيط هادئ — عمق واستقرار	المستثمرون طويلو المدى الذين يفضّلون الهدوء
phantom	شبح خفي — تحليل في الظلال	المتداولون الذين يبحثون عن فرص غير مرئية للآخرين
nova	نجم متفجّر — إبداع وابتكار	المستخدمون الذين يستكشفون استراتيجيات جديدة وغير تقليدية

🔑 الشخصية الافتراضية

`DEFAULT_MOXI_PERSONA` تضبط القيم التالية: الاسم "MOXI"، شخصية ودودة ومتحمسة، خبرات في تحليل العملات الميم وتداول سلاسل الكمية. نمط التواصل "mixed" (مختلط)، ومظهر "default". يمكن للمستخدم تعديل أي من هذه القيم عبر `updateMoxiPersona`.

تُعرّف المتغيرات الثمانية كمصفوفة `AVATAR_VARIANTS` في ملف `persona.ts`، وتُستخدم في مكون `MoxiAvatar` الأمامية لعرض الرمز المناسب. نظام الشخصية مرتبط بقاعدة البيانات عبر `getMoxiPersona(userId)` الذي يسترجع الشخصيات أو يعيد الافتراضية، و `updateMoxiPersona(userId, data)` الذي يحقّط التعديلات.

محرك السياق (Context Engine)

قلب MOXI النابض هو محرك السياق `buildMoxiContext()` الموجود في `context-engine.ts`. هذه الدالة هي جسر تحويل المستخدم المجهول إلى كيان مفهوم تمامًا — حيث تجمع كل البيانات ذات الصلة بالمستخدم في لحظة واحدة قبل إرسالها إلى الذكي. بدون هذا المحرك، سيكون MOXI مجرد روبوت محادثة بلا سياق يردّ بإجابات عامة لا علاقة لها بواقع المستخدم.

يعمل المحرك باستخدام مبدأ **الجلب المتوازي** (Parallel Fetching): عند استدعاء `buildMoxiContext(userId, supabase)` تسعة استعلامات قاعدة بيانات في وقت واحد عبر `Promise.all()`. هذا يعني أن زمن الاستجابة يُحدّد بأبطأ استعلام في المجموعة — فارق كبير في الأداء خاصّةً عند التعامل مع اتصالات شبكة.

المصادر التسعة للسياق

#	المصدر	الوصف	الغرض في الاستجابة
1	profile	ملف المستخدم الأساسي	معرفة مستوى الخبرة وتفضيلات التداول
2	strategy	الاستراتيجية النشطة	تخصيص النصائح حسب نهج المستخدم
3	signalTrackings	الإشارات المُتتَبَعة	معرفة أي إشارات يهتم بها المستخدم حاليًا
4	recentAnalyses	التحليلات الأخيرة	بناءً على السياق التحليلي السابق
5	todaySignals	إشارات اليوم	تقديم رؤية محدثة عن فرص اليوم
6	watchlist	قائمة المراقبة	فهم اهتمامات المستخدم الحالية
7	livePrices	الأسعار المباشرة	تقديم بيانات سعرية حقيقية في الرد
8	recentEvents	الأحداث الأخيرة	ربط الرد بالسياق الزمني الحالي
9	userMemories	ذاكرة المستخدم	تذكّر تفاعلات سابقة وتفضيلات مُعبّر عنها

هيكل الدالة

```
function buildMoxiContext(
  userId: string,
  supabase: SupabaseClient
): MoxiFormattedContext {
  const context: MoxiFormattedContext = {
    profile, strategy, signalTrackings,
    recentAnalyses, todaySignals, watchlist,
    livePrices, recentEvents, userMemories
  };
  await Promise.all([
    getUserProfile(userId, supabase),
    getActiveStrategy(userId, supabase),
    getSignalTrackings(userId, supabase),
    getRecentAnalyses(userId, supabase),
    getTodaySignals(supabase),
    getWatchlist(userId, supabase),
    getLivePrices(supabase),
    getRecentEvents(supabase),
    getUserMemories(userId, supabase),
  ]);
  return { profile, strategy, signalTrackings,
    recentAnalyses, todaySignals, watchlist,
    livePrices, recentEvents, userMemories };
}
```

النتيجة هي كائن من نوع `MoxiFormattedContext` يُمرّر مباشرة إلى `buildMoxiSystemPrompt()` حيث يُدمج في هذا التصميم يفصل بين جلب البيانات وبناء الطلب، مما يسمح باختبار كل جزء بشكل مستقل وتعديل مصادر البيانات دون المنطق الذكي.

⚠️ أهمية السياق في تداول العملات الميم

تداول عملات الميم على سولانا يتطلب فهماً فورياً للسياق. الإشارة التي كانت ممتازة قبل ساعة قد تكون خطيرة الآن. محرك السياق يضمن يرى الصورة الكاملة في كل لحظة — من محفظة المستخدم إلى آخر حدث في السوق.

كل دالة جلب فرعية تتعامل مع حالات الفشل بأمان — إذا تعدّر جلب مصدر معين، يُعيّد بقيمة `null` دون إفشال العملية بأكملها. أن MOXI يستمر في العمل حتى لو كانت بعض الخدمات غير متاحة، مع تقليل جودة السياق بشكل متناسب.

نظام الأدوات (Tool System)

MOXI لا يقتصر على توليد النصوص — يمتلك سجل أدوات متكامل (MOXI_TOOLS) في `tools.ts` يمنحه القدرة على حقيقة في النظام. هذه الأدوات تُفعل عبر `processWithAgent` في طبقة الذكاء (P1) حيث يُكتشف قصد المستخدم ويُنفَّذ الفرق بين MOXI مع أدوات وبدونها هو الفرق بين محلل يُقدّم المشورة ومُنفَّذ يتخذ الإجراء.

سجل الأدوات يحتوي على **سبع أدوات** تغطي الدورة الكاملة لتداول العملات الميم: من التحليل والمراقبة إلى التنبيهات والملخص تُعرّف باسمها ووصفها ومعاملاتها (parameters) وأنواعها، مما يسمح لنظام اكتشاف القصد بفهم متى وكيف يُستدعى كل أداة.

الأداة	الوصف	المعاملات
<code>analyzePair</code>	تحليل شامل لزوج تداول محدد — يشمل السعر والحجم والسيولة والزخم	<code>pairAddress</code> (string, مطلوب)
<code>trackSignal</code>	تتبع إشارة تداول محددة وإضافتها إلى لوحة المتابعة	<code>signalId</code> (string, مطلوب)
<code>createPriceAlert</code>	إنشاء تنبيه سعري عند مستوى محدد لزوج تداول	<code>pairAddress</code> (string), <code>targetPrice</code> (number), <code>direction</code> ('above' 'below')
<code>getMarketSummary</code>	ملخص شامل لسوق عملات الميم على سولانا	لا يوجد معاملات
<code>getPortfolio</code>	استرجاع ملخص المحفظة الحالية للمستخدم	لا يوجد معاملات (يستخدم <code>userId</code> من السياق)
<code>getWatchlist</code>	عرض قائمة المراقبة مع حالة كل عملة	لا يوجد معاملات (يستخدم <code>userId</code> من السياق)
<code>getSignalStatus</code>	التحقق من حالة إشارة تداول محددة	<code>signalId</code> (string, مطلوب)

كيف يعمل اكتشاف القصد: عندما يُرسل المستخدم رسالة مثل "حلّل لي BONK" أو "أضف تنبيه عند 0.00005"، يمرّ `processWithAgent` الذي يُحلّل القصد ويُطابقه مع الأدوات المتاحة. إذا وُجد تطابق، يُنفَّذ الأداة ويُدمج نتيجتها في سياق MOXI. إذا لم يُكتشف قصد أداة، يُحوّل الطلب مباشرة إلى مسار الذكاء الاصطناعي العام.

⚡ أدوات vs محادثة عامة

"ما رأيك بسوق اليوم؟" → محادثة عامة (بدون أدوات).
"حلّل لي BONK/W SOL" → `analyzePair` يُنفَّذ ثم يُبنى الرد حول النتيجة.
"نّبهي إذا وصل SOL لـ \$200" → `createPriceAlert` يُنفَّذ.
هذا التمييز يمنع استدعاء أدوات غير ضرورية ويحسن سرعة الاستجابة.

الأدوات التي لا تحتاج معاملات (`getMarketSummary` ، `getPortfolio` ، `getWatchlist`) تُستدعى تلقائيًا في بعض الحالات، مثلًا، عندما يسأل المستخدم عن حالته العامة، قد يستدعي MOXI `getPortfolio` و `getMarketSummary` معًا لتقديم دون أن يطلب المستخدم كل شيء صراحة.

مولّد الأوامر (Prompt Builder)

في `prompt.ts` ، تقع الدالة `buildMoxiSystemPrompt(persona, ctx)` — وهي المسؤولة عن بناء الطلب الذكي يُرسل إلى مزوّد اللغة الكبيرة. هذه الدالة هي الجسر بين بيانات المستخدم الخام والذكاء الاصطناعي: تأخذ شخصية MOXI وتنتج سلسلة نصية مُهيكلّة تُخبر النموذج بدقّة من هو وماذا يفعل وماذا يعرف.

هيكل الطلب الذكي: يُبنى الطلب من أربعة أقسام متتالية، كل قسم يضيف طبقة من التخصيص:

- قسم الهوية (Identity Block):** يُعرّف MOXI باسمه وشخصيته ونمط تواصله وخبراته. هذا القسم يأتي من كائن `persona` ويُسكّل نبذة الرد الأساسية. إذا كان نمط التواصل "رسمي" يستخدم صياغة احترافية، وإذا كان "عادي" يستخدم لغة مألوفة.
- قسم السياق (Context Block):** يُضخّم كائن `MoxiFormattedContext` الذي أعده محرك السياق. يشمل ذلك ملف استراتيجيته، إشاراته، تحليلاته، أسعار السوق المباشرة، قائمة مراقبته، الأحداث الأخيرة، وذاكرته. كل مصدر يُنسّق بشكل مقرر الطلب.
- قسم القواعد (Rules Block):** تعليمات سلوكية محددة: كيف يتعامل مع الأسعار، كيف يُحدّر من المخاطر، متى يقترح أفعالاً (MOXI_QUICK_ACTIONS)، وكيف يُصيغ الإجابات بشكل مهيكّل.
- قسم الأدوات (Tools Block):** وصف سجل الأدوات المتاحة (MOXI_TOOLS) مع تعليمات حول متى وكيف يُشير إلى استخدام القسم لا يُنفذ الأدوات بل يُعلم النموذج بوجودها لتقديم إجابات تتناسب مع قدراته.

مقتطف من هيكل الطلب

```
buildMoxiSystemPrompt(
  persona: MoxiPersona,
  ctx: MoxiFormattedContext
): string {
  // 1. الهوية (Identity)
  const identity = `
    أنت: persona.name, {persona.personality}
    خبراتك: {persona.expertise.join(', ')}
    نمط التواصل: persona.communication_style
  `

  // 2. السياق (Context)
  const context = `
    استراتيجية: {JSON.stringify(ctx.strategy)}
    قائمة المراقبة: {ctx.watchlist?.map(w => w.pair).join(', ')}
    إشارات: ctx.signalTrackings?.length ?? 0
    ذاكرة المستخدم: {ctx.userMemories?.map(m => m.content).join(', ')}
    الأسعار الحية: {JSON.stringify(ctx.livePrices)}
  `

  // 3. القواعد (Rules)
  const rules = `
    كن دقيقًا بالأرقام واذكر المصدر
    نبيه من المخاطر دائمًا
    اقترح إجراءات سريعة عند الاقتضاء
  `

  return identity + context + rules
}
```

يوجد أيضًا `MOXI_QUICK_ACTIONS` — مصفوفة من الإجراءات السريعة المُعرّفة في `types.ts` التي يُمكن لـ MOXI استخدامها كإجراءات مثل "تتبع هذه الإشارة" أو "إنشاء تنبيه سعري" تُقدّم كأزرار تفاعلية في واجهة المستخدم، مما يُحوّل النص الذكي إلى إجراءات واحدة.

🔑 مبدأ الفصل

فصل بناء الطلب (Prompt) عن محرك السياق (Context) يعني أنه يمكن تعديل كيف يُقدّم السياق للنموذج دون تغيير كيف يُجمع. هذا فصل نماذج اللغة المختلفة قد تحتاج تنسيقات مختلفة لنفس البيانات.

SEC-06

الاستجابة الاستباقية (Proactive Insights)

أحد أكثر جوانب MOXI تميّزًا هو قدرته على الوصول إلى المستخدم قبل أن يطلب. في `notification-hub.ts`، يعمل نظام كمراقب دائم يُحلّل حالة المستخدم ويُولّد رؤى تُنقل عبر `MoxiProactiveInsight`. هذا النظام يُحوّل MOXI من أداة ردّ فَعَلٍ نشط يُدرِك المشاكل قبل أن تتفاقم.

النظام يحتوي على ثلاث وظائف كشف أساسية، كل واحدة تراقب نمطًا مختلفًا من المخاطر:

1. كشف التعرّض المفرط — detectOverexposure

تُحلّل هذه الدالة توزيع المحفظة عبر العملات المختلفة وتكتشف ما إذا كان المستخدم مُعرّضًا بشكل مفرط لعملية واحدة أو قطاع و عملات الميم المتقلّب، التركيز الزائد على عملة واحدة يعني خطرًا كبيرًا إذا انعكست حركتها فجأة. تُقارن الدالة نسبة كل عملة بعتبات مُحدّدة وتُولّد تنبيهًا إذا تجاوزت أي نسبة الحد الآمن.

2. كشف قرب الإشارة — detectSignalProximity

تُراقب هذه الدالة إشارات التداول النشطة وتكتشف التي اقتربت من نقاط التنفيذ أو الانتهاء. مثلًا، إذا كانت إشارة شراء عند مستوى واقترب السعر منه، تُنبّه الدالة المستخدم بأن اللحظة المناسبة اقتربت. هذا يضمن أن المستخدم لا يفوت الفرص بسبب الاندفاع المراقبة المستمرة.

3. كشف مخاطر الأحداث — detectEventRisk

تُحلّل هذه الدالة الأحداث القادمة (مثل إطلاق عملات جديدة، أحداث على السلسلة، تغييرات في السيولة) وتُقيّم تأثيرها المحتمل للمستخدم المفتوحة. إذا كان هناك حدث قد يؤثر سلبيًا على عملة في المحفظة، تُولّد الدالة تحذيرًا استباقيًا مع توضيح المخاطر المحتملة.

```
الهيكل العام - ication-hub.ts
function generateMoxiInsights(userId: string, supabase) {
  const context = await buildMoxiContext(userId, supabase);
  const insights: MoxiProactiveInsight[] = [];
```

```
  // فحص التعرّض المفرط
  const overexposure = await detectOverexposure(context);
  if (overexposure) insights.push(overexposure);
```

```
  // فحص قرب الإشارة
  const proximity = await detectSignalProximity(context);
  if (proximity) insights.push(proximity);
```

```
  // فحص مخاطر الأحداث
  const eventRisk = await detectEventRisk(context);
  if (eventRisk) insights.push(eventRisk);
```

```
  return insights;
```

⚠ التوازن بين الاستباقية والإزعاج

يتم ضبط عتبات الكشف بعناية لتجنب إغراق المستخدم بالتنبيهات. الهدف هو تنبيه ذكي وليس إشعارات عشوائية. كل رؤية استباقية تمرّ بالاهمية قبل إرسالها.

تُستدعى `generateMoxiInsights` بشكل دوري أو عند تغيّر مهم في البيانات (مثل تحديث سعر كبير أو حدث جديد). النتائج يمكن الوصول إليها عبر الدالة الخادمية `getMoxiInsights` (GET) التي تعرض الرؤى الاستباقية الحالية للمستخدم في واجهة

SEC-07

البث المباشر والذكاء الاصطناعي

تجربة المستخدم مع MOXI تعتمد كليًا على البث المباشر (Streaming) عبر نقطة النهاية `/api/copilot-stream`. بدلاً من انتظار دفعة واحدة، يُرسل MOXI الكلمات والجمل تدريجيًا عبر بروتوكول SSE (Server-Sent Events). هذا يمنح بالاستجابة ويُحسّن تجربة الانتظار بشكل كبير خاصةً مع نماذج اللغة الكبيرة التي قد تستغرق ثوانٍ عدة.

عند طلب البث، يُحدّد الوكيل عبر المعامل `agent: "moxi"` الذي يُوجّه الطلب إلى مسار MOXI المحدد. المسار يبدأ بالتحقق ثم بناء السياق ثم توجيه الطلب الذكي، وكل جزء من الرد يُبث فورًا كما يُولّد.

سلسلة مزوّدي الذكاء الاصطناعي (LLM Fallback Chain)

MOXI يستخدم `LLMRouter` لتوجيه الطلبات عبر سلسلة احتياطية من أربعة مزوّدين. هذا يضمن أن MOXI يعمل دائمًا حتى المزوّدين غير متاح أو مُثقلًا:

الأولوية	المزود	الوصف
P1	<code>zai</code>	المزود الأساسي — مُحسّن لفيكسور، أداء أعلى وتخصيص أعمق للسياق المالي
P2	<code>anthropic</code>	Claude — احتياطي أول، ممتاز في التحليل المالي والنصوص الطويلة
P3	<code>groq</code>	Groq — احتياطي ثاني، سرعة فائقة في التوليد
P4	<code>openai</code>	OpenAI — الاحتياطي الأخير، GPT models كخيار نهائي

عند فشل المزود الأساسي (زمن استجابة طويل أو خطأ أو تجاوز الحصة)، ينتقل الجهاز تلقائيًا إلى المزود التالي في السلسلة المستخدم. هذا الانتقال شفاف تمامًا — المستخدم يرى MOXI يردّ بشكل طبيعي بغض النظر عن المزود الفعلي الذي يُعالج الطلب.

نظام الحدود (Rate Limiting)

يُطبّق حدّ معدل صارم: 25 طلبًا في الدقيقة لكل مستخدم عبر `SlidingWindowLimiter`. هذا يحمي النظام من الإساءة عادةً للموارد. نافذة الانزلاق تعني أن الحد يُحسب على آخر 60 ثانية وليس على حدود دقيقة ثابتة — مما يمنع تجاوز الحد بإرسال عند بداية كل دقيقة.

```
import { SlidingWindowLimiter } from 'sliding-window-limiter';
import { AskMoxiRequest } from './types';
import { buildMoxiContext } from './context';
import { buildMoxiSystemPrompt } from './system-prompt';
import { LLMRouter } from './llm-router';

const RATE_LIMIT = { maxRequests: 25, windowMs: 60_000 };

const limiter = new SlidingWindowLimiter(RATE_LIMIT);

async function askMoxi(request: AskMoxiRequest) {
  // التحقق من الحدود
  limiter.check(request.userId, RATE_LIMIT);

  // بناء السياق (9 مصادر بالتوازي)
  const ctx = await buildMoxiContext(request.userId, supabase);

  // بناء الطلب الذكي
  const systemPrompt = buildMoxiSystemPrompt(persona, ctx);

  // إرسال الطلب عبر LLMRouter مع agent: "moxi"
  const response = await LLMRouter.stream({ systemPrompt, messages, agent: 'moxi' });
}
```

⚠ تجاوز الحدود

عند تجاوز 25 طلبًا/دقيقة، يُرفض الطلب فورًا مع رمز الحالة 429. لا يوجد طابور انتظار — المستخدم يحتاج الانتظار حتى تنخفض معدّلته. إغراق مزوّدي الذكاء الاصطناعي بالطلبات.

SEC-08

قاعدة بيانات MOXI

يعتمد MOXI على جدول `moxi_personas` في قاعدة بيانات Supabase لتخزين شخصيات المستخدمين. هذا الجدول هو الحديقة للشخصيات المخصصة، ويضمن أن تفضيلات المستخدم تُحفظ وتُسترجع بشكل متسق عبر الجلسات والأجهزة.

العمود	النوع	القيود	الوصف
user_id	UUID	PRIMARY KEY, FK → profiles.id	معرّف المستخدم — يرتبط بجدول الملفات الشخصية
name	TEXT	'DEFAULT 'MOXI	اسم الشخصية — الافتراضي "MOXI"
personality	TEXT	NOT NULL	وصف نصي للشخصية يُمرَّر في الطلب الذكري
expertise	JSONB	'[]' DEFAULT	مصفوفة مجالات الخبرة ([]string)
communication_style	TEXT	CHECK ('formal','casual','mixed')	نمط التواصل مع القيم المسموحة فقط
avatar_variant	TEXT	CHECK (8 variants)	متغير المظهر من القيم المعرّفة
nft_token_id	TEXT	NULLABLE	معرّف رمز NFT إن ارتبطت الشخصية برمز غير موجود للاستبدال
is_customized	BOOLEAN	DEFAULT false	علامة تُشير إلى ما إذا كانت الشخصية قد عُدِّلَت الافتراضية

العلاقة مع جدول `profiles`: `user_id` يعمل كمفتاح خارجي يرتبط بجدول `profiles`. هذا يضمن أن كل شخصية MOXI شخصية صالح وأن حذف الملف الشخصي يؤدي لحذف الشخصية تلقائيًا (CASCADE). عند إنشاء مستخدم جديد، لا تُنشأ سجل شخصية — بل يُستخدم `DEFAULT_MOXI_PERSONA` من الكود حتى يقوم المستخدم بتخصيص شخصيته لأول مرة.

سياسات أمان الصفوف (RLS)

- SELECT**: المستخدم يستطيع قراءة شخصيته فقط (`user_id = auth.uid()`)
- INSERT**: المستخدم يستطيع إنشاء شخصية لحسابه فقط
- UPDATE**: المستخدم يستطيع تعديل شخصيته فقط
- DELETE**: مقيّد — لا يمكن حذف الشخصية يدويًا (يتم عبر `cascade` عند حذف الملف)

استراتيجية التخزين

`expertise` يُخزَّن كـ JSONB وليس كجدول منفصل لأن عدد المجالات محدود (3-6 عادةً) ولا تحتاج استعلامات معقدة. هذا يُبسّط القراءة مع الحفاظ على المرونة في إضافة مجالات جديدة.

المُشغَّلات (Triggers): عند الإدراج الأول لسجل شخصية، يُفَعَّل مُشغِّل يُعيّن `is_customized = false`. عند أي تحديث تلقائيًا إلى `true`، مما يسمح للنظام بمعرفة ما إذا كان يجب استخدام الشخصية المخزّنة أم الافتراضية، ويُسهّل ميزة "إعادة الافتراضي" ببساطة.

الدوال الخادمية المتعلقة بقاعدة البيانات

- `getMoxiPersona(userId)` — تستعلم `moxi_personas` وتُعيد الشخصية أو `DEFAULT_MOXI_PERSONA`
- `updateMoxiPersona(userId, data)` — تُحدّث السجل وتُعيّن `is_customized = true`
- `getMoxiPersonaFn` — دالة خادمية (GET) تُغلّف `getMoxiPersona` للواجهة الأمامية
- `updateMoxiPersona` — دالة خادمية (POST) تُغلّف `updateMoxiPersona` للواجهة الأمامية

واجهات MOXI في المنتج

MOXI ليس مجرد كود خلفي — يظهر في عدة نقاط عبر واجهة فيكسور الأمامية كعنصر بصري وتفاعلي حي. كل نقطة تكامل مُصممة بطبيعية وغير مُزعجة، بحيث يشعر المستخدم بأن MOXI جزء أصيل من التجربة وليس إضافة خارجية.

مكون MoxiAvatar

مكون `MoxiAvatar` هو التمثيل البصري لـ MOXI في الواجهة. يستقبل خاصية `variant` من نوع `AvatarVariant` الرمز المناسب. المتغيرات الثمانية تُترجم إلى ألوان وتأثيرات وحركات مختلفة: `default` باللون البنفسجي الأساسي لفيكسور، بالأخضر الصاعد، `bear` بالأحمر الهابط، `crystal` بالأزرق الشفاف، `flame` بالبرتقالي المتقدّ، `ocean` بالأزرق العميق، بالرمادي الشفاف، و `nova` بالبنفسجي المتوهّج.

المكون يدعم ثلاثة أحجام: `sm` (في القوائم والأزرار)، `md` (في رأس المحادثة)، و `lg` (في الشاشات الترحيبية). كل حجم ليتناسب مع سياق الاستخدام دون أن يطغى على المحتوى المحيط.

واجهة محادثة MOXI

محادثة MOXI تظهر كجزء من نظام الـ Copilot في فيكسور. عند فتح نافذة المحادثة، يُحدّد الوكيل كـ `"moxi"` مما يُفعلّ الكامل: بناء السياق، اكتشاف القصد، التوجيه عبر مزوّد الذكاء الاصطناعي، والبث المباشر. واجهة المحادثة تعرض:

- فقاات الرسائل:** رسائل المستخدم والردود الذكية بتنسيق Markdown مع دعم الكود والجدول
- الإجراءات السريعة:** أزرار `MOXI_QUICK_ACTIONS` التي يقترحها MOXI (مثل "تتبع الإشارة" أو "إنشاء تنبيه")
- مؤشر الكتابة:** مؤشر بث مباشر يُظهر أن MOXI يُولّد الرد في الوقت الحقيقي
- الرؤى الاستباقية:** بطاقات `MoxiProactiveInsight` التي تظهر في أعلى المحادثة عند وجود تنبيهات

نقاط التكامل عبر التطبيق

الموقع	التكامل	الوصف
الشريط الجانبي	أيقونة MoxiAvatar	رمز MOXI في الشريط الجانبي يفتح نافذة المحادثة بنقرة واحدة
لوحة الإشارات	زر "اسأل MOXI"	عند كل إشارة، زر يفتح المحادثة مع سياق الإشارة المُحدّدة
بطاقة العملة	تكامل السياق	عند فتح بطاقة عملة، يُضاف السياق تلقائيًا لمحادثة MOXI
الإشعارات	رؤى استباقية	تنبيهات MOXI الاستباقية تظهر في مركز الإشعارات مع رمز MOXI
الصفحة الرئيسية	ترحيب MOXI	رسالة ترحيب شخصية من MOXI تُحدّث حسب حالة السوق وملف المستخدم

🎯 التكامل غير المُزعج

مبدأ التصميم هو أن MOXI يظهر عندما يحتاجه المستخدم ويختفي عندما لا يحتاجه. لا توجد نوافذ منبثقة إجبارية أو إشعارات مُزعجة — التفاعل بالمبادرة من المستخدم أو عبر تنبيهات استباقية مُضبطة بعناية.

خريطة التتبع: MOXI → الكود

هذا الفصل هو الخريطة الشاملة التي تربط كل ميزة MOXI بمسار الملف الدقيق واسم الدالة. استخدم هذه الخريطة للتنقل مباشر من كود MOXI دون بحث.

الميزة	الملف	الدالة / التصدير
نوع الشخصية	src/domains/moxi/types.ts	MoxiAvatarVariant , MoxiPersona
نوع الاستجابة	src/domains/moxi/types.ts	MoxiResponse
نوع الرؤية الاستباقية	src/domains/moxi/types.ts	MoxiProactiveInsight
الإجراءات السريعة	src/domains/moxi/types.ts	QUICK_ACTIONS , MoxiQuickAction
نوع السياق المُنسَّق	src/domains/moxi/types.ts	MoxiFormattedContext
الشخصية الافتراضية	src/domains/moxi/types.ts	DEFAULT_MOXI_PERSONA
التصديرات المجمعّة	src/domains/moxi/index.ts	Barrel — كل الأنواع والأدوات والدوال
سؤال MOXI (API)	src/domains/moxi/functions.ts	(POST, 25 req/min) askMoxi
تحديث الشخصية (API)	src/domains/moxi/functions.ts	(POST) updateMoxiPersona
جلب الشخصية (API)	src/domains/moxi/functions.ts	(GET) getMoxiPersonaFn
جلب الرؤى (API)	src/domains/moxi/functions.ts	(GET) getMoxiInsights
محرك السياق	src/domains/moxi/context-engine.ts	ldMoxiContext(userId, supabase)
مولّد الطلب الذكي	src/domains/moxi/prompt.ts	ldMoxiSystemPrompt(persona, ctx)
إدارة الشخصية (DB)	src/domains/moxi/persona.ts	ldMoxiPersona , getMoxiPersona
متغيرات المظهر	src/domains/moxi/persona.ts	AVATAR_VARIANTS (8 متغيرات)
سجل الأدوات	src/domains/moxi/tools.ts	MOXI_TOOLS (7 أدوات)
أداة تحليل الزوج	src/domains/moxi/tools.ts	analyzePair
أداة تتبع الإشارة	src/domains/moxi/tools.ts	trackSignal
أداة التنبيه السعري	src/domains/moxi/tools.ts	createPriceAlert
أداة ملخص السوق	src/domains/moxi/tools.ts	getMarketSummary
أداة المحفظة	src/domains/moxi/tools.ts	getPortfolio
أداة قائمة المراقبة	src/domains/moxi/tools.ts	getWatchlist
أداة حالة الإشارة	src/domains/moxi/tools.ts	getSignalStatus
كشف التعرّض المفرط	src/domains/moxi/notification-hub.ts	detectOverexposure
كشف قرب الإشارة	src/domains/moxi/notification-hub.ts	detectSignalProximity
كشف مخاطر الأحداث	src/domains/moxi/notification-hub.ts	detectEventRisk
توليد الرؤى الاستباقية	src/domains/moxi/notification-hub.ts	generateMoxiInsights
البث المباشر (SSE)	src/app/api/copilot-stream/route.ts	agent: "moxi"
مكون المظهر	واجهة المستخدم	MoxiAvatar (variant, size)

الميزة	الملف	الدالة / التصدير
جدول قاعدة البيانات	Supabase Migration	moxi_personas (RLS + Triggers)

ملخص الملفات

- 6 — types.ts أنواع + 2 ثوابت افتراضية
 - index.ts — تصدير مجمع لكل شيء
 - 4 — functions.ts دوال خادمية (3 GET + 1 POST)
 - 1 — context-engine.ts دالة رئيسية + 9 دوال جلب فرعية
 - 1 — prompt.ts دالة بناء الطلب الذكي
 - 2 — persona.ts دالة DB + 1 مصفوفة متغيرات
 - 1 — tools.ts سجل أدوات (7 أدوات)
 - 3 — notification-hub.ts دوال كشف + 1 دالة توليد
- المجموع: 8 ملفات، 25+ دالة/تصدير

وثيقة رسمية لمنظومة فيكسور الهندسية

VIXOR — AI-Powered Trading Terminal